EP5. PDHG and cuPDLP for large-scale linear programs

Lidivine Kengne | IE 5311-001 (D01), Homework 5

1. Why a first-order method for LP at all. Both mature methods solve linear systems. Simplex keeps a factorization of the basis A_B and updates it; the interior point method forms and factors $A\Theta A^\top$ every iteration. That is where the memory goes: even for sparse A the factors fill in, and the fill is not under the modeller's control. At hundreds of millions of nonzeros the factors stop fitting, and the solver fails for reasons unrelated to how many iterations it would have needed.

2. The main idea of PDHG. Take $\min c^\top x$ subject to $Ax \geq b$, $x \geq 0$. Its Lagrangian, with $y \geq 0$ on the rows, is $L(x, y) = c^\top x + y^\top (b - Ax)$, and strong duality makes the LP equivalent to the saddle point problem $\min_{x \geq 0} \max_{y \geq 0} L(x, y)$. This is the Lagrangian derived in Problem 2 Part 5; the difference is that there x was minimised out to leave a dual, whereas here *neither* side is eliminated. Primal–dual hybrid gradient takes one projected gradient step in each variable, extrapolating the primal:

$$x^{k+1} = \mathrm{proj}_{x \geq 0}\big(x^k - \tau(c - A^\top y^k)\big), \quad y^{k+1} = \mathrm{proj}_{y \geq 0}\big(y^k + \sigma\big(b - A(2x^{k+1} - x^k)\big)\big).$$

Projection onto an orthant or a box is a componentwise max, so an iteration costs exactly one Ax and one $A^\top y$: $O(\mathrm{nnz}(A))$ work and $O(\mathrm{nnz}(A))$ memory, no fill-in ever, no linear system anywhere. Convergence needs $\tau\sigma\|A\|_2^2 < 1$ and the averaged iterates close the primal–dual gap at rate $O(1/k)$. The term $2x^{k+1} - x^k$ is not cosmetic: without it this is Arrow–Hurwicz, which need not converge.

3. From PDHG to PDLP. Textbook PDHG on a real LP stalls at one or two digits, so the algorithm above is necessary but nowhere near sufficient. PDLP is the engineering that closes the gap:

Restarting	Restart periodically from the running average. On an LP the normalised duality gap is *sharp*, which turns the $O(1/k)$ rate into a linear one. The single largest gain.
Primal weight	Set $\tau = \eta/\omega$, $\sigma = \eta\omega$ and adapt ω, so primal and dual residuals fall together.
Adaptive step	Replace the global, pessimistic bound $\eta < 1/\|A\|_2$ by a step accepted or rejected on local curvature.
Presolve and scaling	Ruiz equilibration and diagonal preconditioning; conditioning of A sets the constant in every bound above.

4. How cuPDLP applies this at scale. cuPDLP is PDLP moved onto a GPU, and the reason it works is that the iteration was already the right shape. Each step is two sparse matrix–vector products plus componentwise arithmetic and a few reductions for the restart and step-size tests — all data-parallel, all bandwidth-bound, which is precisely what GPU memory systems deliver. Nothing in the method is sequential: there is no pivot order to respect, no factorization to build, no triangular solve. That is the contrast with the alternatives, since a simplex pivot sequence is inherently serial and a sparse Cholesky parallelises only within its elimination tree while filling in badly. Memory tells the same story: the method holds A in sparse form plus a handful of vectors of length n and m, so instances whose *factors* would exhaust host memory have a *footprint* that fits on a single device. Large and sparse is therefore the regime where the method is strongest, and it is why first-order LP moved from curiosity to practice and has since appeared in commercial solvers.

5. What it costs. Accuracy: relative gaps of 10^{-4} to 10^{-8} are routine and 10^{-9} is hard, where simplex returns an exact vertex. No basis: hence no warm start for branch and bound and no clean sensitivity analysis, the duals coming out approximate — which matters for exactly the shadow-price reading done in Homework 1. And predictability: performance tracks conditioning in a way that is hard to forecast from the model. On small and medium LPs, where a factorization is cheap, simplex and interior point remain correct; the case for PDHG begins where the factors stop fitting.